МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка

ЗВІТ

до лабораторної роботи №6
з дисципліни «Професійний Python»

Тема: «Автоматизоване тестування Python-програм»

Виконав:
студент Левандівський Назар

Перевірив:
Куньо Іван Михайлович

Львів – 2026

Мета роботи

Сформувати навички професійного автоматизованого тестування Python-програм засобами pytest: unit та integration tests, fixtures, параметризація, перевірка винятків, ізоляція файлової системи й середовища (tmp_path, monkeypatch), test doubles (Mock, AsyncMock, fake) та аналіз statement і branch coverage.

1. Об'єкт тестування

Варіант 1. Система обліку студентів. Тестується проєкт `student_manager` з ЛР1 (студент із полями `first_name`, `last_name`, `group`, `average_grade`; функції `calculate_group_average`, `find_best_student`, `filter_by_group`), доповнений пошуком за ID, репозиторіями, сповіщеннями, CSV import, JSON export, YAML-конфігурацією й async-сервісом.

Таблиця 1 – Структура production code (src/student_manager)

Модуль	**Відповідальність**
`models.py`	`Student` (frozen dataclass), `classify_grade`, 4 рівні успішності
`validators.py`	перевірка й розбір ID, імен, групи й середнього бала 0–100
`services.py`	`filter_by_group`, `find_by_last_name`, `calculate_group_average`, `find_best_student`; `StudentService`, `AsyncStudentService`, `Notifier` (Protocol)
`repositories.py`	`StudentRepository` і `AsyncStudentRepository` (Protocol), in-memory реалізації
`importers.py`	CSV import із валідацією кожного рядка
`exporters.py`	JSON export через тимчасовий файл (атомарно)
`reports.py`	зведений звіт: групи, найкращий, рівні, ризик
`config.py`	YAML + змінні `STUDENT_MANAGER_*`
`main.py`	потік CSV → StudentService → JSON, CLI

2. Структура test suite і test matrix

Таблиця 2 – Файли тестів і кількість тестів (з параметризацією)

Файл	**Тестів**
`integration/test_application_flow.py`	11
`integration/test_export_pipeline.py`	5
`integration/test_import_pipeline.py`	4
`unit/test_async_services.py`	9
`unit/test_configuration.py`	34
`unit/test_exporters.py`	9
`unit/test_importers.py`	26
`unit/test_models.py`	42
`unit/test_reports.py`	10
`unit/test_repositories.py`	11
`unit/test_services.py`	45
`unit/test_validators.py`	82
Разом	**288**

Спільні fixtures винесено в `tests/conftest.py`, fake і spy – у `tests/support.py`. Integration tests позначено маркером `integration` (зареєстрований у `pyproject.toml`, `--strict-markers`), тому їх можна запускати окремо.

Таблиця 3 – Відповідність пунктів варіанта 1 файлам тестів

Пункт варіанта 1	**Файли тестів**
створення Student, обов'язкові поля	test_models, test_validators
пошук за прізвищем або ID	test_services, test_repositories
фільтрація за групою	test_services
обчислення середнього бала	test_services
найкращий студент	test_services, test_async_services
порожня колекція	test_services, test_reports
boundary та invalid середній бал	test_validators, test_models
CSV import, malformed record	test_importers
JSON export	test_exporters, test_export_pipeline
YAML/configuration	test_configuration
StudentRepository: Mock або fake	test_services, support.py
Notifier: Mock та interaction	test_services
збій залежності через side_effect	test_services, test_async_services
AsyncMock, потік імпорт → звіт	test_async_services, test_application_flow

Таблиця 4 – Test matrix

Категорія	**Що перевіряється (приклади)**
Normal	реєстрація, пошук за ID та прізвищем, фільтр за групою, середній бал групи, найкращий студент, потік CSV → JSON
Min boundary	середній бал 0, 0.01 і −0.0 (нормалізується до 0.0); рівень unsatisfactory для 0; група з одного студента
Max boundary	середній бал 99.99 і 100; межі рівнів 59.99/60, 74.99/75, 89.99/90; середній бал рівно на порозі сповіщення (60)
Empty	порожня колекція (`NoStudentsError`), звіт для порожньої колекції, CSV лише із заголовком
Invalid type/value	бал −0.01, 100.01, NaN, ∞, текст, bool; ID 0, −5, bool, «1»; порожні імена; некоректна група; 12 видів malformed CSV
Missing data	немає config, ключів input_path/output_path, CSV, колонок; порожній файл; файл не в UTF-8
Dependency failure	Notifier → `NotificationError`, сховище → `RepositoryError`, async-сховище → `ConnectionError`, `json.dump` → `OSError`, `os.replace` → `PermissionError`
Repeated operation	повторний student_id (у сховищі й у CSV); повторний запуск потоку перезаписує звіт
State transition	невдале оновлення балу не змінює студента й не надсилає сповіщення; збій сповіщення не скасовує реєстрацію; збій export не псує попередній файл

3. Fixtures

Використано autouse (`_clean_environment` прибирає змінні `STUDENT_MANAGER_*`), scope=session (незмінний набір груп), параметризовану fixture (`csv_encoding`: кодування з BOM і без), fixture factory (`student_factory` з лічильником ID), `Mock(spec=Notifier)`, ланцюжок fixtures (`service` →

`populated_service`) і fixture із yield `output_dir`, яка після тесту перевіряє, що атомарний export не лишив тимчасових файлів.

```
@pytest.fixture(autouse=True)
def _clean_environment(monkeypatch: pytest.MonkeyPatch) -> None:
    """Прибирає змінні середовища застосунку, щоб тести не залежали від ОС."""
    for name in ENV_NAMES:
        monkeypatch.delenv(ENV_PREFIX + name, raising=False)

@pytest.fixture(scope="session")
def allowed_groups() -> frozenset[str]:
    """Незмінний набір дозволених груп; безпечно ділити на всю сесію."""
    return frozenset({"PI-21", "PI-22", "PI-23"})

@pytest.fixture(params=["utf-8", "utf-8-sig"])
def csv_encoding(request: pytest.FixtureRequest) -> str:
    """Параметризована fixture: кодування з BOM і без нього."""
    return str(request.param)

@pytest.fixture
def student_factory() -> StudentFactory:
    """Фабрика студентів: параметри за замовчуванням, унікальні id."""
    counter = itertools.count(1)

    def create(
        last_name: str = "Коваленко",
        first_name: str = "Олена",
        group: str = "PI-21",
        average_grade: float = 90.0,
        student_id: int | None = None,
    ) -> Student:
        identifier = next(counter) if student_id is None else student_id
        return Student(identifier, first_name, last_name, group, average_grade)

    return create
```

```
@pytest.fixture
def notifier() -> Mock:
    """Mock із spec: дозволені лише методи протоколу Notifier."""
    return Mock(spec=Notifier)

@pytest.fixture
def output_dir(tmp_path: Path) -> Iterator[Path]:
    """Каталог експорту; після тесту перевіряє, що не лишилось *.tmp."""
    directory = tmp_path / "output"
    yield directory
    leftovers = list(directory.glob("*.tmp")) if directory.exists() else []
    assert not leftovers, f"тимчасові файли не видалено: {leftovers}"
```

4. Приклади unit tests

Parametrization і boundary test: межі рівнів оцінювання (з `ids` для читабельного виводу).

```
@pytest.mark.parametrize(
    ("average", "level"),
    [
        (0, "unsatisfactory"),
        (59.99, "unsatisfactory"),
        (60, "satisfactory"),
        (74.99, "satisfactory"),
        (75, "good"),
        (89.99, "good"),
        (90, "excellent"),
        (100, "excellent"),
    ],
    ids=[
        "min",
        "below-60",
        "exactly-60",
        "below-75",
        "exactly-75",
        "below-90",
        "exactly-90",
        "max",
    ],
```

```
)
def test_classify_grade_boundaries(average: float, level: str) -> None:
    assert classify_grade(average) == level
    assert level in LEVELS
```

pytest.raises, custom exception, exc_info і pytest.approx: середнє порожньої групи піднімає `NoStudentsError`; помилка бала знає поле (`field`); середнє від чисел із плаваючою комою порівнюється з допуском.

```
def test_group_average_of_empty_collection_raises_custom_error() -> None:
    with pytest.raises(NoStudentsError, match="без студентів"):
        calculate_group_average([])

def test_group_average_tolerates_floating_point_noise(
    student_factory: StudentFactory,
) -> None:
    students = [student_factory(average_grade=g) for g in (0.1, 0.2)]
    assert calculate_group_average(students) != 0.15
    assert calculate_group_average(students) == pytest.approx(0.15)

def test_invalid_grade_error_names_field_and_offending_value() -> None:
    with pytest.raises(InvalidGradeError, match="101") as error:
        validate_average_grade(101)
    assert error.value.field == "average_grade"
    assert isinstance(error.value, ValueError)
```

Mock, return_value, interaction assertion:

```
def test_register_saves_student_and_notifies_once(
    service: StudentService, notifier: Mock, student_factory: StudentFactory
) -> None:
    student = student_factory(group="PI-22")
    service.register(student)
    assert service.get(student.student_id) is student
    notifier.send.assert_called_once_with(
        student, "Вас зараховано до групи PI-22"
    )

def test_service_reads_student_through_mock_repository(
    student_factory: StudentFactory,
) -> None:
    student = student_factory(student_id=7)
    repository = create_autospec(StudentRepository, instance=True)
    repository.get.return_value = student
    service = StudentService(
        repository, create_autospec(Notifier, instance=True)
    )
    assert service.get(7) is student
    repository.get.assert_called_once_with(7)
```

side_effect і assert_not_called: збій сповіщення не скасовує реєстрацію, а збій сховища не допускає сповіщення.

```
def test_notifier_failure_does_not_roll_back_registration(
    service: StudentService,
    notifier: Mock,
    student_factory: StudentFactory,
    caplog: pytest.LogCaptureFixture,
) -> None:
    notifier.send.side_effect = NotificationError("SMTP недоступний")
    student = student_factory(student_id=6)
    with caplog.at_level(logging.WARNING):
        service.register(student)
    assert service.get(6) is student
    assert service.failed_notifications == [6]
    assert "Не вдалося сповістити студента 6" in caplog.text

def test_repository_failure_prevents_notification(
    student_factory: StudentFactory,
) -> None:
    repository = create_autospec(StudentRepository, instance=True)
    notifier = create_autospec(Notifier, instance=True)
    repository.add.side_effect = RepositoryError("сховище недоступне")
    service = StudentService(repository, notifier)
```

```
    with pytest.raises(RepositoryError):
        service.register(student_factory())
    notifier.send.assert_not_called()
    assert not service.failed_notifications
```

monkeypatch і tmp_path: змінна середовища перевизначає YAML; підміна `json` у модулі експорту перевіряє, що атомарний export не псує попередній файл.

```
def test_environment_variables_override_yaml(
    config_writer: ConfigWriter, monkeypatch: pytest.MonkeyPatch
) -> None:
    path = config_writer(pass_threshold="60")
    monkeypatch.setenv("STUDENT_MANAGER_PASS_THRESHOLD", "75")
    monkeypatch.setenv("STUDENT_MANAGER_SKIP_INVALID", "no")
    monkeypatch.setenv("STUDENT_MANAGER_OUTPUT_PATH", "elsewhere/result.json")
    config = load_config(path)
    assert config.pass_threshold == pytest.approx(75.0)
    assert config.skip_invalid is False
    assert config.output_path.name == "result.json"

def test_failed_write_keeps_previous_file_and_leaves_no_temporary_file(
    output_dir: Path, monkeypatch: pytest.MonkeyPatch
) -> None:
    target = output_dir / "report.json"
    write_json(target, {"version": 1})

    def broken_dump(*args: object, **kwargs: object) -> None:
        raise OSError("диск переповнено")

    monkeypatch.setattr(exporters, "json", SimpleNamespace(dump=broken_dump))
    with pytest.raises(ExportError, match="report.json") as error:
        write_json(target, {"version": 2})
    assert isinstance(error.value.__cause__, OSError)
    assert json.loads(target.read_text(encoding="utf-8")) == {"version": 1}
```

5. AsyncMock

Асинхронний репозиторій – зовнішня залежність, тому `AsyncStudentService` тестується з `AsyncMock(spec=AsyncStudentRepository)`. Перевіряються результат, `assert_awaited_once_with`, `assert_not_awaited` і збій мережі (`side_effect=ConnectionError`), що стає `RepositoryError` зі збереженою причиною. Режим pytest-asyncio – `strict`, кожен async-тест має `@pytest.mark.asyncio`.

```
@pytest.fixture
def async_repository() -> AsyncMock:
    """AsyncMock за інтерфейсом AsyncStudentRepository."""
    return AsyncMock(spec=AsyncStudentRepository)

@pytest.mark.asyncio
async def test_load_returns_student_from_async_repository(
    async_service: AsyncStudentService,
    async_repository: AsyncMock,
    student_factory: StudentFactory,
) -> None:
    student = student_factory(student_id=1)
    async_repository.get.return_value = student
    assert await async_service.load(1) is student
    async_repository.get.assert_awaited_once_with(1)

@pytest.mark.asyncio
async def test_connection_failure_becomes_repository_error_with_cause(
    async_service: AsyncStudentService, async_repository: AsyncMock
) -> None:
    async_repository.get.side_effect = ConnectionError("таймаут")
    with pytest.raises(RepositoryError, match="сховище недоступне") as error:
        await async_service.load(1)
    assert isinstance(error.value.__cause__, ConnectionError)
```

6. Integration tests

Integration tests працюють із реальними файлами в `tmp_path` і справжніми компонентами (config → CSV import → валідація → StudentService → report → JSON export); замінено лише зовнішню межу – `Notifier`. Async-тест поєднує імпорт із `AsyncStudentService`.

```
@pytest.mark.usefixtures("output_dir")
def test_pipeline_writes_json_report(
    csv_writer: CsvWriter, config_writer: ConfigWriter
) -> None:
    csv_writer(*STUDENT_ROWS)
    config = load_config(config_writer(allowed_groups="[PI-21, PI-22, PI-23]"))
    report = run_pipeline(config, Mock(spec=Notifier))
    saved = json.loads(config.output_path.read_text(encoding="utf-8"))
    assert saved == report
    assert saved["total_students"] == 5
    assert saved["best_student"]["student_id"] == 3
    assert saved["at_risk"] == [4]
    assert [e["line"] for e in saved["import_errors"]] == [7, 8]
    assert saved["import_errors"][1]["message"] == "група PI-24 не дозволена"
```

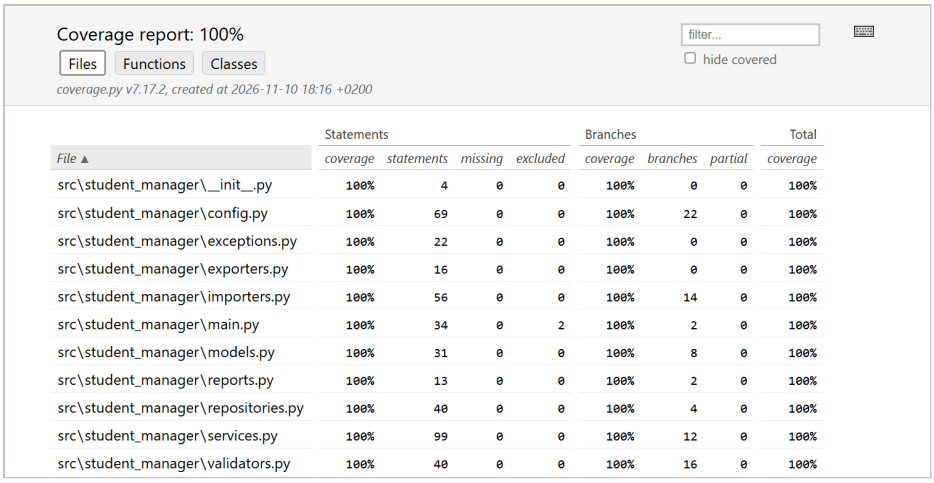
7. Результати запуску та coverage

```
PS C:\Github\prof_python\Lab_06> pytest -q --cov=student_manager --cov-branch --cov-report=term-missing --
cov-report=html
........................................................................ [ 25%]
........................................................................ [ 50%]
........................................................................ [ 75%]
........................................................................ [100%]
============================== tests coverage ==============================
_______________ coverage: platform win32, python 3.12.10-final-0 _______________

Name                                  Stmts   Miss Branch BrPart  Cover   Missing
---------------------------------------------------------------------------------
src\student_manager\__init__.py           4      0      0      0   100%
src\student_manager\config.py            69      0     22      0   100%
src\student_manager\exceptions.py        22      0      0      0   100%
src\student_manager\exporters.py         16      0      0      0   100%
src\student_manager\importers.py         56      0     14      0   100%
src\student_manager\main.py              34      0      2      0   100%
src\student_manager\models.py            31      0      8      0   100%
src\student_manager\reports.py           13      0      2      0   100%
src\student_manager\repositories.py      40      0      4      0   100%
src\student_manager\services.py          99      0     12      0   100%
src\student_manager\validators.py        40      0     16      0   100%
---------------------------------------------------------------------------------
TOTAL                                   424      0     80      0   100%
Coverage HTML written to dir htmlcov
Required test coverage of 90.0% reached. Total coverage: 100.00%
288 passed in 1.77s
```

Рисунок 1 – Запуск усього suite: 288 passed, 0 failed, coverage 100 % (statement і branch)

Набір	**Результат**
`pytest -m "not integration"` (unit)	268 passed, 20 deselected in 0.64s
`pytest -m integration`	20 passed, 268 deselected in 0.64s



Coverage report: 100%

Files | Functions | Classes

filter... | hide covered

coverage.py v7.17.2, created at 2026-11-10 18:16 +0200

	Statements				Branches			Total
File ▲	coverage	statements	missing	excluded	coverage	branches	partial	coverage
src\student_manager__init__.py	100%	4	0	0	100%	0	0	100%
src\student_manager\config.py	100%	69	0	0	100%	22	0	100%
src\student_manager\exceptions.py	100%	22	0	0	100%	0	0	100%
src\student_manager\exporters.py	100%	16	0	0	100%	0	0	100%
src\student_manager\importers.py	100%	56	0	0	100%	14	0	100%
src\student_manager\main.py	100%	34	0	2	100%	2	0	100%
src\student_manager\models.py	100%	31	0	0	100%	8	0	100%
src\student_manager\reports.py	100%	13	0	0	100%	2	0	100%
src\student_manager\repositories.py	100%	40	0	0	100%	4	0	100%
src\student_manager\services.py	100%	99	0	0	100%	12	0	100%
src\student_manager\validators.py	100%	40	0	0	100%	16	0	100%

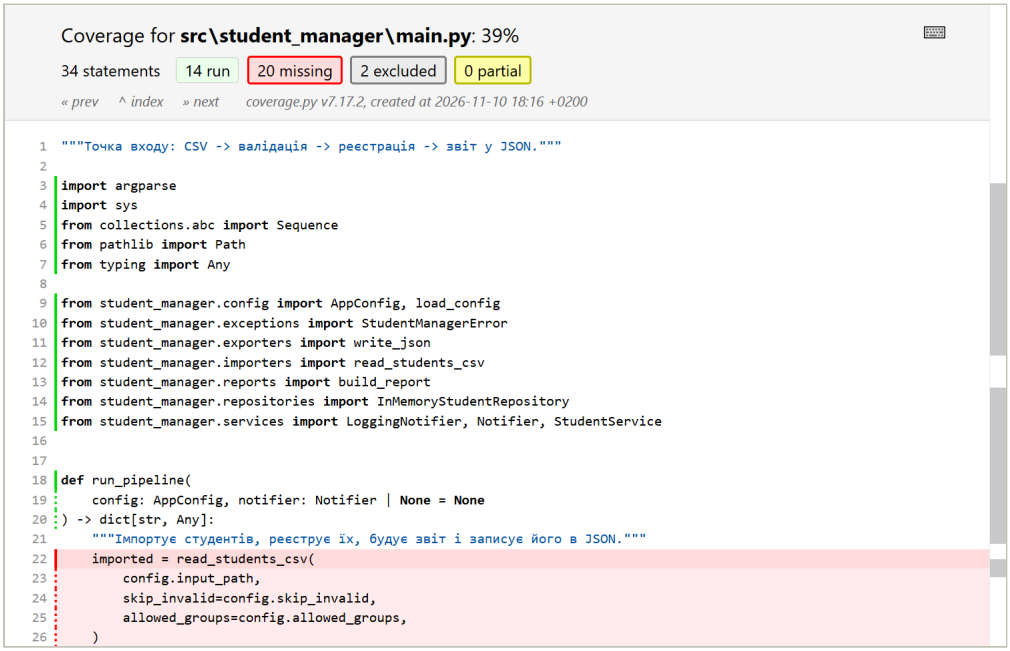
Рисунок 2 – HTML coverage report (htmlcov/index.html), скриншот Microsoft Edge

Колонка «excluded» (2 рядки) – це блок `if __name__ == "__main__"` (правило `exclude_also` у pyproject.toml): виконуваного коду в ньому немає. Усі 80 гілок виконано в обох напрямках (колонка BrPart = 0).

8. Аналіз недостатньо протестованих paths і дефекти

Missing lines. Якщо запустити лише unit tests, загальне покриття падає до 96 %. Пропущені рядки показують, які шляхи перевіряють лише integration tests:

```
PS C:\Github\prof_python\Lab_06> pytest -m "not integration" --cov=student_manager --cov-branch   #
фрагмент: файли без повного покриття і TOTAL
Name                                Stmts   Miss Branch BrPart  Cover   Missing
-------------------------------------------------------------------------------
src\student_manager\main.py            34     20      2      0    39%   22-42, 47-65
-------------------------------------------------------------------------------
TOTAL                                 424     20     80      0    96%
```



Coverage for **src\student_manager\main.py**: 39%

34 statements | 14 run | 20 missing | 2 excluded | 0 partial

« prev ^ index » next coverage.py v7.17.2, created at 2026-11-10 18:16 +0200

```
1  """Точка входу: CSV -> валідація -> реєстрація -> звіт у JSON."""
2
3  import argparse
4  import sys
5  from collections.abc import Sequence
6  from pathlib import Path
7  from typing import Any
8
9  from student_manager.config import AppConfig, load_config
10 from student_manager.exceptions import StudentManagerError
11 from student_manager.exporters import write_json
12 from student_manager.importers import read_students_csv
13 from student_manager.reports import build_report
14 from student_manager.repositories import InMemoryStudentRepository
15 from student_manager.services import LoggingNotifier, Notifier, StudentService
16
17
18 def run_pipeline(
19     config: AppConfig, notifier: Notifier | None = None
20 ) -> dict[str, Any]:
21     """Імпортує студентів, реєструє їх, будує звіт і записує його в JSON."""
22     imported = read_students_csv(
23         config.input_path,
24         skip_invalid=config.skip_invalid,
25         allowed_groups=config.allowed_groups,
26     )
```

Рисунок 3 – Сторінка main.py у HTML coverage report після запуску лише unit tests: червоним позначено рядки, яких тести не виконали

Рядки 22-42, 47-65 у `main.py` – функції `run_pipeline` і `main`: їх виконує лише повний потік (integration tests, CLI). Додавати unit tests немає потреби. Навпаки, лише integration tests дають 83 % і не перевіряють boundary та invalid-вхід, тому обидва види tests потрібні.

Coverage не гарантує відсутність дефектів. Перший запуск suite (266 tests) дав 100 % statement і branch coverage й жодного падіння (фрагмент виводу):

```
PS C:\Github\prof_python\Lab_06> pytest -q --cov=student_manager --cov-branch --cov-report=term-missing
# перший запуск, фрагмент
Name                                  Stmts   Miss Branch BrPart  Cover   Missing
TOTAL                                   420      0     76      0   100%
Required test coverage of 90.0% reached. Total coverage: 100.00%
266 passed in 0.86s
```

Після цього додано 22 негативні tests на суворий формат числа, −0.0, NaN/∞ та циклічні структури в export, нетекстові шляхи в YAML. Покриття лишилось 100 %, але 19 tests упали (і 1 error у teardown) – tests виявили п'ять реальних дефектів production code:

```
PS C:\Github\prof_python\Lab_06> pytest   # до виправлень; фрагмент short test summary info, [N] – кількість параметризованих випадків
ERROR tests/unit/test_exporters.py::test_circular_structure_raises_export_error_and_keeps_old_file
FAILED tests/unit/test_configuration.py::test_non_text_path_raises_configuration_error  [8]
FAILED tests/unit/test_exporters.py::test_non_finite_number_is_not_exported_as_invalid_json  [2]
FAILED tests/unit/test_exporters.py::test_circular_structure_raises_export_error_and_keeps_old_file
FAILED tests/unit/test_validators.py::test_parse_average_grade_rejects_everything_but_plain_decimals  [7]
FAILED tests/unit/test_validators.py::test_negative_zero_is_stored_as_positive_zero
19 failed, 269 passed, 1 error in 1.82s
```

Таблиця 5 – Дефекти production code, знайдені tests

№	Симптом	Причина	Виправлення
D1	`parse_average_grade` приймає `+90`, `-0`, `1e2`, `9_0`, `٩` (арабсько-індійські цифри), `.5`, `5.` (`DID NOT RAISE InvalidGradeError`)	розбір через `float()`, який дозволяє знак, експоненту, підкреслення й будь-які Unicode-цифри	суворий шаблон `[0-9]+(\.[0-9]+)?`
D2	`validate_average_grade(-0.0)` повертає `-0.0`; у JSON з'явився б «-0.0»	`-0.0 >= 0` – істина, знак нуля не нормалізовано	`return grade + 0.0` (−0.0 → 0.0)
D3	`write_json` записує `NaN` і `Infinity` – це не валідний JSON (`DID NOT RAISE ExportError`)	`json.dump` за замовчуванням має `allow_nan=True`	`allow_nan=False`, помилка стає `ExportError`
D4	циклічна структура → необроблений `ValueError`, а файл `report.json.tmp` залишається на диску	`except` ловив лише `OSError` і `TypeError`	`ValueError` додано до `except`: тимчасовий файл видаляється
D5	`input_path: [a.csv]`, `123`, `true` приймаються як шлях «['a.csv']», «123» (`DID NOT RAISE ConfigurationError`)	значення YAML довільного типу перетворювалось на рядок через `str()`	`parse_path` вимагає рядок, інакше `ConfigurationError`

Після виправлень усі 288 tests проходять, покриття лишилось 100 %. Висновок: відсоток coverage показує лише виконаність коду; дефекти знайшли tests із осмисленими assertions для invalid-входу.

9. Експерименти

Таблиця 6 – Експерименти 1–2: слабкий і сильний тест на «мутанті» (середнє замінено на максимум)

Тест	Код без помилки	Мутант	Coverage services.py
`test_average_weak` (без assert)	passed	passed	36.0 %
`test_average_strong`	passed	failed	36.0 %

Обидва тести виконують однакові рядки й дають однаковий coverage, але помилку ловить лише тест з assert: execution ≠ verification.

Таблиця 7 – Експерименти 3–7

Експеримент	Результат
3. Branch coverage `classify_grade`	лише excellent і unsatisfactory: 4/6 гілок (66.7 %), пропущено переходи 20→21, 22→23 (рівні good і satisfactory); з усіма чотирма рівнями: 6/6 (100.0 %)
4. Mock vs fake	після еквівалентного рефакторингу (`StudentService.get` через `all_students()` замість `repository.get()`): тест з Mock – failed, з fake – passed
5. Flaky test	випадкові ID студентів: 3 падінь із 20 запусків; з фіксованим seed: 0 із 20
6. tmp_path isolation	3 запуски – різні тимчасові каталоги; змінених файлів у репозиторії: 0
7. Failure path	success і failure залежностей: Notifier (`return` / `side_effect=NotificationError`), сховища (`RepositoryError`, `ConnectionError`), export (`OSError`); перевірено і виняток, і відсутність небажаних взаємодій (`assert_not_called`, `assert_not_awaited`)

Mock перевіряє взаємодію і тому прив'язаний до реалізації – ламається при рефакторингу без зміни поведінки; fake перевіряє стан, легше читається й перевикористовується, але потребує власного коду. Mock доречний на системних межах (сповіщення, мережа), fake – для сховищ. Flaky-тест небезпечний тим, що підриває довіру до червоних збірок; випадковість треба фіксувати (seed, ін'єкція даних).

Висновки

Створено test suite із 288 тестів для системи обліку студентів (варіант 1): unit tests моделі, валідаторів, сервісів, репозиторіїв, звітів, конфігурації, CSV import і JSON export, async-сервісу та integration tests потоку CSV → валідація → звіт, CLI і async-потоку. Використано fixtures різних видів, параметризацію, pytest.raises, approx, tmp_path, monkeypatch, Mock/autospec, AsyncMock і fake. Негативні tests знайшли п'ять дефектів production code, яких не бачив 100 % coverage; statement і branch coverage – 100 % при порозі 90 % у pyproject.toml. Експерименти показали, що coverage не замінює змістовних assertions, а детермінованість та ізоляція тестів важливіші за їх кількість.